that it's not `null` and not a duplicate of another element already in our *Universe*.

Now we get to really interesting stuff: first and foremost, the `findPartition` method, shown in listing 5.3.

Listing 5.3 Naïve solution, findPartition

```
function
findPartition(elem)
      #1
  throw-if (elem == null or not
partitionsMap.has(elem))                    #2
  return
partitionsMap[elem]
      #3
```

#1 Takes an element and return a `Set`: the partition (aka disjoint set) to which the element belongs.
#2 Check that the element is valid.
#3 Return the `Set` containing the argument.

In this basic implementation, the method is particularly trivial: after the usual validation (including checking that the element is actually stored in the disjoint-set) we just need to return the partition containing `elem`.

As we mentioned, this implementation of the `findPartition` method only requires constant time (assuming that the hash for `elem` can be computed in constant time).

Another easy to implement method is the one checking if two elements belong to the same partition.